Field Hours

# CSCE 712 Digital Forensic Engineering

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

## Scope and the custody log

You open a case by writing the question, the devices in scope, and the time window. A chain of custody is the mechanism that ties a later image to that seizure. Each handoff records 4 fields: the item, who released it, who received it, and the clock time. If you leave one field blank, a reader cannot show that item 2 on the bench is the drive you took from the desk.

You photograph connectors before you unplug them, because cable order is part of the scene. Then you mark the device with a case number and an item number, such as item 3. The bag, the form, and the image you make later must carry that same pair. A report about a laptop you cannot match to item 3 is a story. It is not an exhibit you can defend.

Four fields on each handoff

Item identifier

Person who released it

Person who received it

Time of the handoff

### Item 3 changes hands twice

1. You seize item 3 at 09:10 and log yourself as the receiver. The releaser is the person who had the laptop.

2. At 11:40 you hand item 3 to a colleague. The log now has a second line: you release, they receive, time filled in.

3. The image made that afternoon is labeled item 3. A reader can walk both lines without guessing who held the drive.

Problem 1. The log for item 3 names the releaser and the receiver but leaves the time blank. What can you not claim?

Answer. You cannot claim a complete chain for item 3. The handoff needs 4 fields, and the time is one of them. Names alone do not show when the drive moved.

Problem 2. You photographed the ports, labeled item 5, and then imaged a different unlabeled laptop. What is wrong with the exhibit?

Answer. The image is not tied to item 5. The bag, the form, and the image must carry the same item number. A careful photo does not repair a swapped device.

Lecture 2 · 90 minutes

# Imaging through a write blocker

You acquire a disk through a write blocker, which forwards read commands and drops writes. The result is a forensic image plus a hash of the sectors you copied. You compute the same hash, often SHA-256, on the source and on the image, and you write both digests in the notes. If they differ by 1 bit, you do not search that image. You acquire it again.

A copy of visible files is not enough when you still need deleted space. The image keeps unallocated clusters, slack, and the partition table. You store it on wiped media and you keep a second copy under the same item number. NIST tool tests check whether a product reports a known image, including a planted mismatch. You read 1 such report before you trust a tool you have not used.

From device to checked image

Attach the write blocker -> Copy every sector -> Hash source and image -> Store two labeled copies

## A four-sector toy image

1. The source has 4 sectors. The blocker accepts the reads and rejects a write your imager tries by mistake.

2. You hash the 4 sectors and hash the image. The two digests match, so this image is the one you will search.

3. You keep a second copy on other media, still labeled with the item number. A later scratch does not force a new seizure.

Problem 1. The source digest and the image digest differ. What do you do with that image?

Answer. You do not search it. A difference of even 1 bit means the image is not a faithful copy. You acquire the source again through the write blocker and compare the hashes once more.

Problem 2. You copied only the files a user can see, and the hashes of those files match. Is deleted space preserved?

Answer. No. A logical file copy never stored unallocated clusters or slack. Matching file hashes do not replace a full image when the question needs deleted space.

Lecture 3 · 90 minutes

# Names, clusters, and carved candidates

A file system maps names to clusters through metadata. When you delete a file, many systems only clear a flag in the directory entry. The clusters can sit unchanged until a later write reuses them. You may carve a header, such as a 4-byte signature of a known type, from unallocated space. You label that hit a candidate, not a restored user file with a trustworthy name.

You do not treat a recovered name as proof of who typed the bytes. Account identifiers, link files, and logs have to agree with the name. A thumbnail can show a picture whose main file is already gone. You record the path, which of 2 timestamps you trusted, and the tool version. 3 agreeing artifacts beat 1 parser you cannot explain to a

reader.

Candidate versus named file

Carved candidate

Header found in free space

No trustworthy path

Say it is a candidate

Named file

Directory entry still points

Path and owner recorded

Corroborate with logs

### One deleted note, one thumbnail

1. The directory flag is clear, but 2 clusters still hold a 4-byte header you recognize. You carve those clusters and mark them a candidate.

2. A thumbnail in another folder shows the same picture. You record both places and you do not invent a user name for the carved bytes.

3. A log on the same image shows the account that owned the folder. You cite the log, the thumbnail, and the candidate as 3 separate facts.

Problem 1. Your carver prints a file name next to a header found in free space. What do you call the result, and why?

Answer. You call it a candidate. Deletion often clears the directory flag and leaves clusters behind. A header is not a path, and a path the tool guessed is not proof of who wrote the bytes.

Problem 2. You have a carved header, a thumbnail, and an account log that all fit the same picture. How many artifacts are you citing?

Answer. You are citing 3 artifacts. They are stronger together than 1 parser, and you still describe each source separately instead of merging them into a single guessed file.

Lecture 4 · 90 minutes

## Clocks and a timeline you can defend

A timestamp is a clock plus a meaning. One file can store a created time, a modified time, and an accessed time, and those 3 fields do not move together. You convert every stamp into one zone, usually UTC, and you keep the raw value beside it. If two sources disagree by 5 seconds, you do not average them. You write down the skew and which source you used.

You anchor the timeline to an event that also appears off the disk, such as a login in a network log. A laptop that was off can drift. If that clock is ahead of UTC, you subtract the gap. A gap of 2 hours means you subtract 2. A

chart with no hypothesis is not a finding. You state the claim, or you do not.

Build one defended timeline

1. Read each raw stamp

2. Convert into UTC

3. Note any skew

4. State one hypothesis

### A clock that is two hours ahead

1. The file's modified stamp reads 11:00 on the laptop. You have already measured that this clock is 2 hours ahead of UTC.

2. You subtract 2 hours. The stamp is 09:00 UTC. You keep 11:00 in the notes as the raw value.

3. A network log shows a related login at 09:20 UTC. The file stamp is 20 minutes earlier. That is the hypothesis you can write, not a guess about intent.

Problem 1. The laptop is 3 hours ahead of UTC. A file stamp reads 16:40 local. What is the UTC time?

Answer. 13:40 UTC. A clock ahead of UTC by H hours is converted by subtracting H, so 16:40 minus 3 hours is 13:40. You still keep the raw 16:40 beside it.

Problem 2. Two tools report the same event 5 seconds apart. Should you average them to 2.5 seconds of adjustment?

Answer. No. You record the 5-second skew and name which source you used. Averaging hides a disagreement the reader needs to see.

Lecture 5 · 90 minutes

## Volatile collection before shutdown

RAM can hold processes, keys, and sockets that a disk image will not contain after you power off. You collect volatile evidence first, and you record that the collection itself changed memory. A memory image is still a copy you hash. The order is fixed: capture RAM, note the process list, then image the disk. Doing those 3 steps backward loses the sockets you came for.

You do not trust a process name by itself. The name is a string the program chose. You read the parent, the path, the user, and the endpoint together. A 16-byte key in a process is a lead, not proof the user typed it. You name the tool and the system profile. Pages the kernel already evicted are a limit you state, not a gap you fill.

Order from volatile to disk

Memory image first

Process list second

Disk image third

### A socket that shutdown would drop

1. The machine is on. You image RAM and hash that image before you ask the box to shut down.

2. The process list shows one process with a network endpoint. You record parent, path, user, and endpoint as 4 fields.

3. Only then do you image the disk. The report says the endpoint came from memory, because the disk image alone would not have kept it.

Problem 1. You imaged the disk, shut the machine down, and only then wished you had the sockets. What did the order cost you?

Answer. The sockets were volatile. After shutdown they are gone, and the disk image cannot recreate them. The required order is RAM, then the process list, then the disk: 3 steps, memory before power off.

Problem 2. A process is named like a system tool and holds a 16-byte key. What may you claim?

Answer. You may claim that this process image contained a 16-byte key, and you must cite parent, path, user, and endpoint. You may not claim the user typed the key, or that the name alone identifies the program.

Lecture 6 · 90 minutes

# Logs and captures are partial

Network evidence is usually a log or a packet capture, and both are partial. A firewall log may store the time, the two addresses, the ports, and the action, and drop the payload. You hash the original file before you filter it. A capture of 10 minutes is not the whole day. You write the start, the stop, and the interface, or a reader cannot see what you never recorded.

You correlate a flow with a host artifact only when the address, the port, and the time agree inside the skew you already measured. If the log shows a query at 14:02 and your capture starts at 14:05, you do not invent the missing packets. You report that the capture cannot confirm the response. A negative finding is still a finding. It keeps the 2 sources distinct.

What each source can show

Firewall log

Addresses, ports, action

Payload often absent

Hash the file first

Packet capture

Bytes on one interface

Only while it was running

Write start and stop

## A query just before the capture

1. The DNS log records a query at 14:02. Your capture file starts at 14:05 and runs 10 minutes, until 14:15.

2. You hash the capture before you filter. Inside the file you do not see that query or its response.

3. You report both facts. You do not fill the 3-minute gap with packets you do not have.

Problem 1. The capture runs from 14:05 to 14:15. The log shows a query at 14:02. Can you describe the response packets?

Answer. No. The query is 3 minutes before the capture starts. You report the log line and the capture window, and you do not invent packets for the gap.

Problem 2. You filtered a firewall log and then hashed the filtered file only. What did you fail to preserve?

Answer. The original log. You hash the original before you filter so a reader can see that your later worksheet came from that file. A hash of the filtered copy does not anchor the source.

Lecture 7 · 90 minutes

# Tool tests and the written report

A tool is evidence about evidence. You name the product, the version, and the settings, because a parser change can move a timestamp. NIST's tool testing program publishes results on known data. You read the test for the function you rely on. If that report says the tool missed 2 of 8 planted cases, you do not write that your single hit is certain.

The report answers the question in the scope, in an order a stranger can check. You separate facts, interpretations, and limits. A fact is that the hash matched and the file was at a path. An interpretation is that the account likely opened it. A limit is that you never imaged the second drive. You attach the custody log. A 1-page conclusion with no appendix is not something you should sign.

From notes to a signed report

Name tool and version -> Separate fact from limit -> Attach the custody log -> Answer only the scope

## A tool that missed two planted files

1. The NIST-style report you read says this version missed 2 of 8 planted deleted files.

2. Your case has one carved candidate. You write that the tool can miss planted files, so absence is not proof the file was never there.

3. The conclusion stays inside the scope, cites the hash, and points at the custody log. The miss rate stays in the limits section.

Problem 1. A tool missed 2 of 8 planted files in a published test. Your image shows no hit. What may you tell the reader?

Answer. You may say the tool reported no hit on this image. You may not say the file was never present. A known miss on 2 of 8 planted cases belongs in the limits.

Problem 2. Your draft says the user intended to hide a file. Which bucket is that sentence in?

Answer. It is an interpretation, not a fact. Facts are things like a matching hash and a path. Intent is not established by a parser output, and the limit section should say what you did not image.

Lecture 8 · 90 minutes

# What you can now do with a case

You can now take a device from a labeled seizure to a report a reader can audit. You can image through a write blocker, require matching hashes, and refuse a copy that differs by 1 bit. You can build a timeline in UTC: if a clock is ahead of UTC by H hours, you subtract H. You can keep the 3-step volatile order and say what a tool failed to show in a published test.

You can also keep the legal half as a procedure rather than a mood. A handoff still needs 4 fields, and an item number must match from bag to image. When a log and a capture disagree, you report both windows instead of inventing packets. Practice on a toy image you create, with 3 planted files and 1 deleted, and write the short report. If you cannot point at the hash, you are not done.

A case you can close

1. Log four custody fields

2. Hash the full image

3. Convert clocks to UTC

4. State facts and limits

## Hash match, then a clock conversion

1. The source hash and the image hash match, so you may analyze this copy. The match does not order events in time.

2. A file stamp reads 11:40 on a laptop you measured as 2 hours ahead of UTC. You subtract 2. The stamp is 09:40 UTC.

3. A log line for the same case sits at 09:10 UTC. The file stamp is 30 minutes later. You report that order and you keep the raw 11:40.

Problem 1. The two hashes match. The laptop is 4 hours ahead of UTC, and a file stamp reads 18:15 local. A log line is at 13:15 UTC. Is the file stamp before or after the log line, and by how many minutes? Use the ahead-of-UTC rule.

Answer. Subtract 4 hours from 18:15 to get 14:15 UTC. The log line is 13:15 UTC, so the file stamp is 60 minutes after the log line. Matching hashes only show the image matches the disk. They do not put the events in order.

Problem 2. Your toy practice image has 3 planted files and 1 deleted file, but the custody form has no time. What must you fix before you sign?

Answer. The handoff is incomplete: you still need all 4 fields, including the time. The planted counts do not replace the custody log, and you also need a hash you can point at.